



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Reverse Engineering an AI-Based System

CSA1709 -Artificial Intelligence

Presented by:

Ganji Madhan Kumar [192472328]

Guided By:

Dr P Rajaram

SIMATS ENGINEERING
CO5- ASSESSMENT TOOL 3 – Technical Presentation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 10%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I Ganji Madha Kumar(192472328) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
 I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:G.Madhan/

1. System Analysis Presentation

Present a reverse engineering analysis of an AI-based system, explaining its architecture, components, and working process using diagrams.

2. Architecture Reconstruction Demo

Prepare a technical presentation to reconstruct the architecture of a given software/AI system, highlighting module interactions and data flow.

3. Algorithm & Logic Identification

Present how you identified algorithms and decision-making logic in a reverse-engineered system, including methodology and tools used.

4. Data Flow & Processing Pipeline

Explain through a presentation the data flow, processing pipeline, and communication mechanisms in a reverse-engineered distributed system.

5. Enhancement & Redesign Proposal

Deliver a technical presentation proposing improvements or redesign strategies for an existing system based on reverse engineering findings.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed, using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

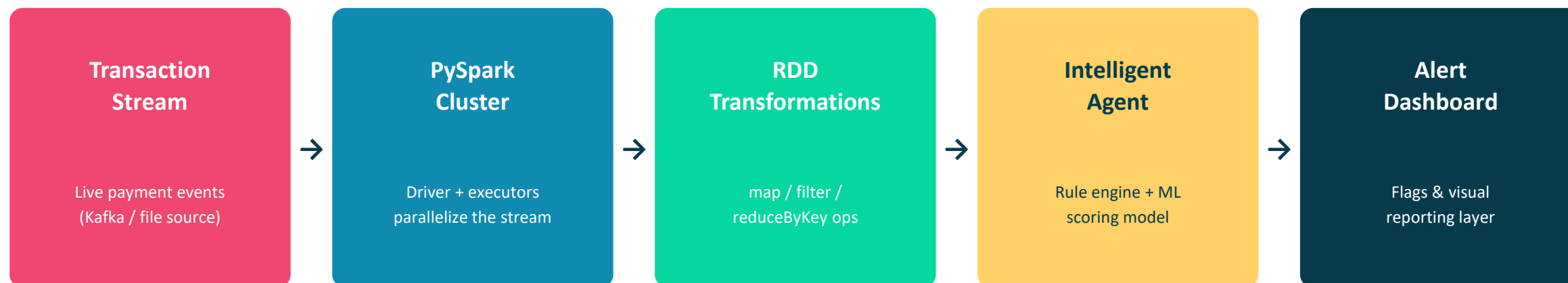


Engineer to Excel



System Analysis: Distributed Fraud Detection System

Reverse-engineering an AI-based system built on PySpark, RDDs, and an intelligent decision agent



ARCHITECTURE STYLE

Distributed, event-driven, layered

CORE FRAMEWORK

Apache Spark (PySpark) on a cluster

DECISION UNIT

Hybrid rule-based + ML scoring agent

Architecture Reconstruction: Module Interactions & Data Flow

Presentation Layer

Dashboard, alerts, REST API for analysts



Agent / Decision Layer

Rule engine + ML classifier module



Processing Layer (PySpark)

RDD transformations, aggregations, joins



Data Ingestion Layer

Kafka consumers, batch file loaders

KEY MODULE INTERACTIONS

- Kafka consumer → `SparkContext.parallelize()` creates the base RDD
- `RDD.filter()` and `.map()` run on executors in parallel partitions
- `reduceByKey()` aggregates per-account transaction features
- Aggregated RDD is collected and passed to the agent module
- Agent publishes flagged events back to the dashboard queue

Data flows upward through each layer; control & feedback signals flow back down to the agent for adaptive thresholds.

Algorithm & Logic Identification

Methodology and tools used to uncover decision-making logic in the reverse-engineered system

1

Static Code Review

Inspected transformation chains in the PySpark job scripts

2

RDD Lineage Tracing

Used `toDebugString()` to reconstruct the DAG of operations

3

Runtime Profiling

Monitored stages/tasks via the Spark Web UI

4

Decision Logic Extraction

Traced agent thresholds and model weights from config & logs

ALGORITHMS IDENTIFIED

● Rule-Based Thresholding

Flags transactions above amount / velocity limits

● Logistic Regression Scorer

Computes a fraud-probability score per transaction

● Random Forest Classifier

Cross-validates borderline cases from the regression stage

● Anomaly Scoring (Z-score)

Detects deviation from a user's historical spending pattern

Data Flow & Processing Pipeline

End-to-end pipeline and communication mechanisms in the distributed system

1 Ingest

Kafka topic streams raw transaction records

2 Parallelize

SparkContext creates a partitioned RDD

3 Transform

map / filter clean & normalize each record

4 Aggregate

reduceByKey groups by account / merchant

5 Shuffle & Broadcast

Shared lookup tables sent via broadcast vars

6 Decide & Emit

Agent scores results, emits alerts to sink

COMMUNICATION MECHANISMS

Kafka topics for ingestion • Spark shuffle for aggregation • Broadcast variables for shared lookups • Accumulators for run metrics • Driver-executor RPC over the cluster manager

Enhancement & Redesign Proposal

Improvement strategies based on reverse-engineering findings

CURRENT LIMITATIONS

- ✗ RDD API lacks Catalyst query optimization
- ✗ Batch-only processing adds latency to alerts
- ✗ No checkpointing — full recompute on failure
- ✗ Static rule thresholds don't adapt over time

PROPOSED REDESIGN

- ✓ Migrate RDDs to the DataFrame / Spark SQL API
- ✓ Adopt Structured Streaming for near real-time alerts
- ✓ Add checkpointing & caching for fault tolerance
- ✓ Introduce a feedback loop so the agent retrains thresholds